

CPSC 330 Lect

Ensemble



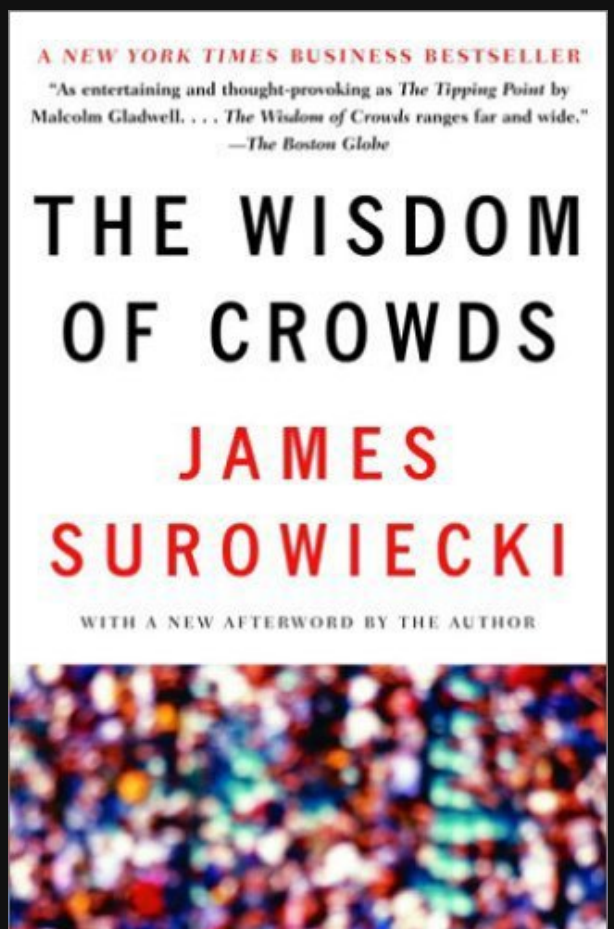
iClicker Exercise 11.0

Which of the following scenarios has worked effectively

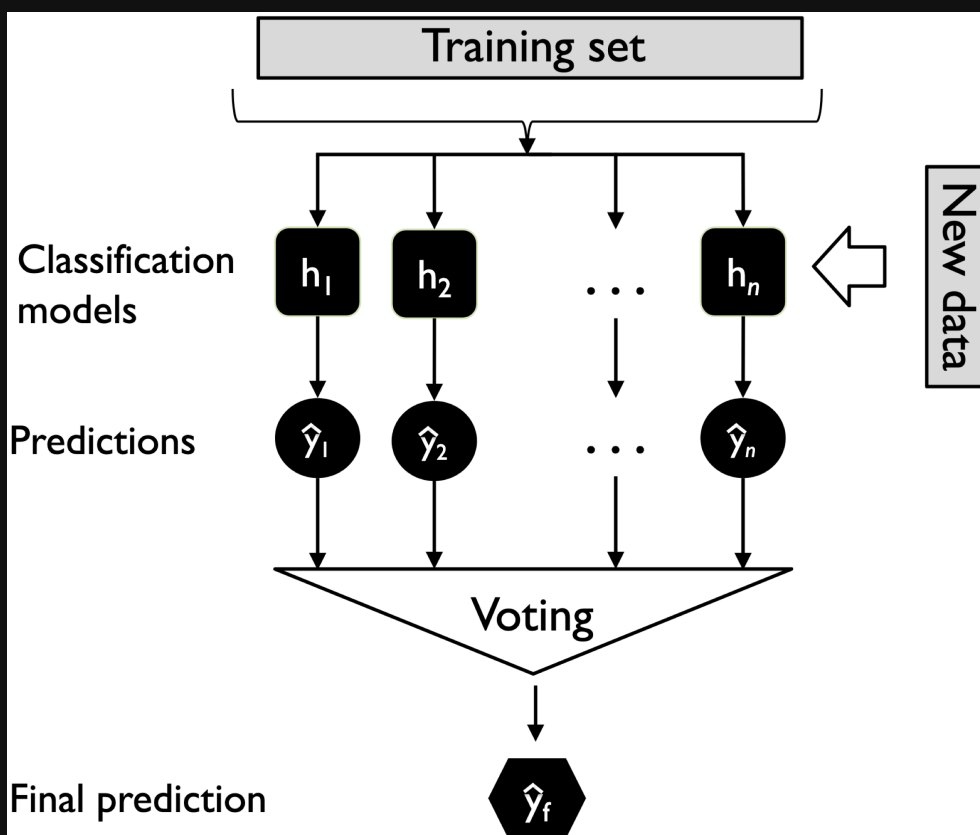
- A. Working independently on a project/assignment.
- B. Working with like-minded people.
- C. Teaming up with a diverse group offering varied p

The Wisdom of Crowds

Groups can often make better decisions than individual members are diverse enough.



Ensembles key idea



- Combine pre-trained models for a more stable result
- Classification: majority class or average probabilities
- Regression: Average (or median) prediction
- Averaging reduces errors, making the ensemble more robust than individual models

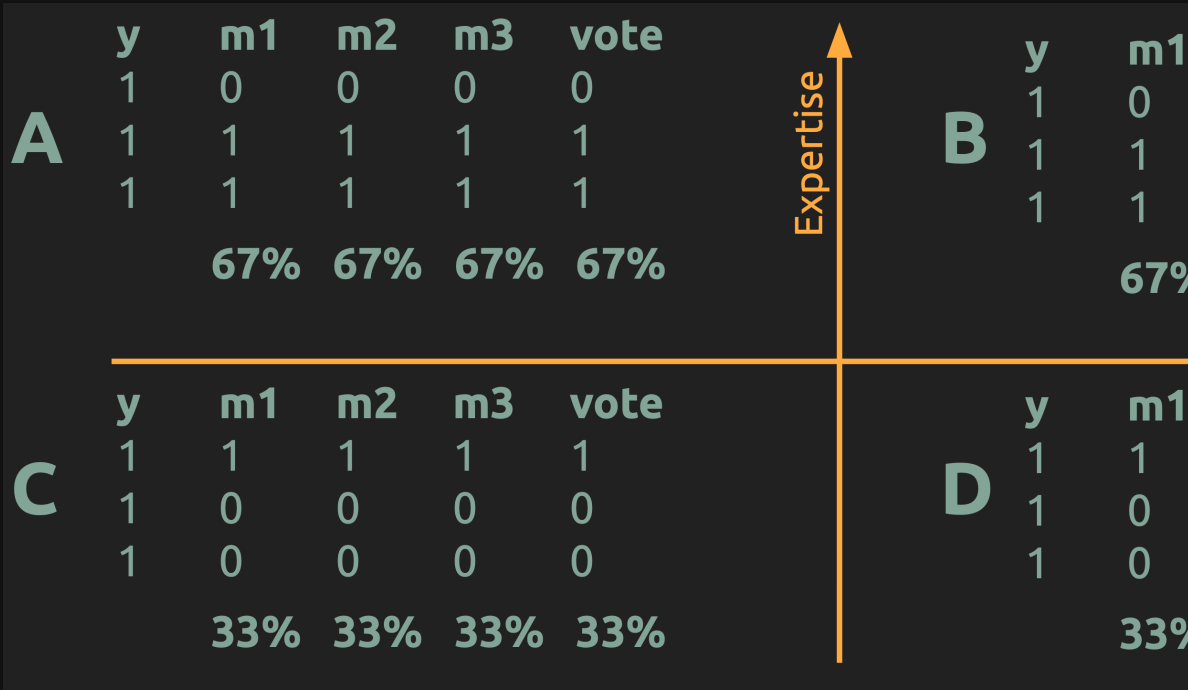
Source

Ensembles

- Ensemble methods are **widely used in industry** and **learning competitions** (e.g., Kaggle).
- In this course, we'll explore
- **Tree-based ensembles:**
 - Random Forests
 - Gradient Boosted Trees (at a high level)
- **Ways to combine models:**
 - **Averaging:** combine model outputs directly
 - **Stacking:** use one model to learn how to combine

When do ensembles h

In which of these scenarios does an ensemble improve



Source

When do ensembles h

Ensembles improve performance when the individual models are **competent** (have expertise) and **make different kinds**

Diversity

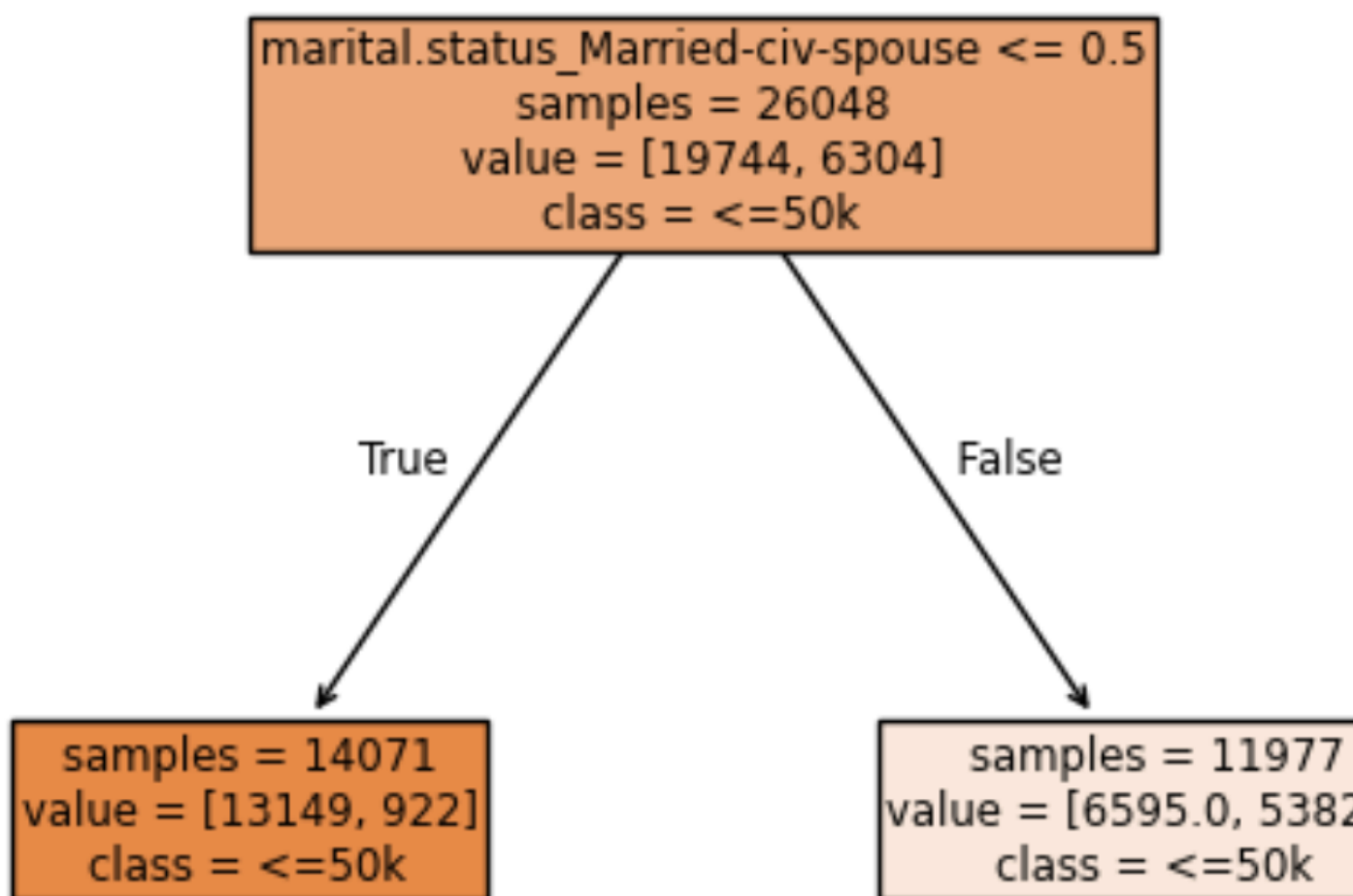
- Ensembles work best when their models make **differ**
- If all models make the same errors, averaging won't h
- We can **encourage diversity** by:
 - Introducing randomness (e.g., bootstrapped sample subsets (*Random Forests*))
 - Sequentially focusing on previous errors (e.g., *Gradient*

Expertise

- Diversity alone is not enough. Individual models should have **predictive skill**.
- If each model performs poorly (worse than random), the ensemble will not recover.
- Best ensembles strike a **balance** between:
 - **Diversity:** models disagree in useful ways
 - **Expertise:** each model performs better than chance

Random forest

What can we randomize between trees?



Two sources of random random forests

- **Bootstrap sampling of data**
Each tree is trained on a slightly different dataset → o
- **Random subset of features at each split**
Each split considers only a random subset of features correlation.

Bootstrap sample

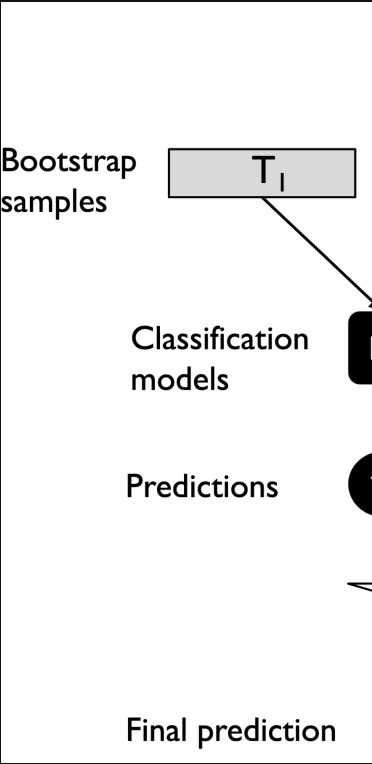
- Ensures trees **see different versions of the data**
- Creates **diversity** while preserving **similar overall st**

Training example indices	Bootstrap sample 1	Bootstrap sample 2	...
1	2	7	...
2	2	3	...
3	1	2	...
4	3	1	...
5	7	1	...
6	2	7	...
7	4	7	...

h_1

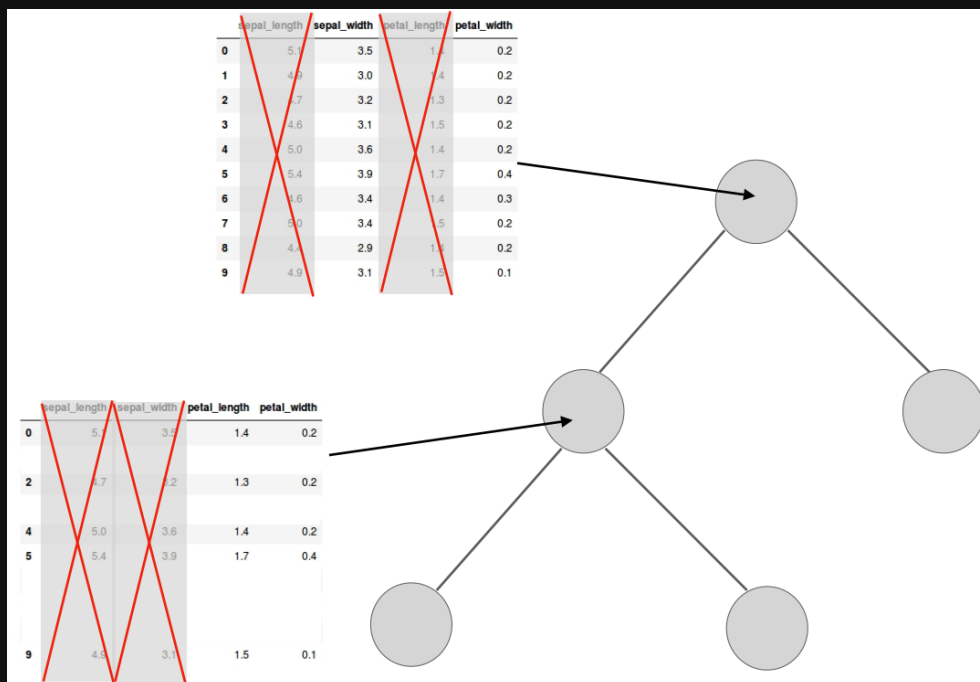
h_2

h_n





Random subset of features at each split



- Select a different random subset of features for every split
- Each tree node splits the data into two parts of the feature space
- 🤔 But why not use all features in a single tree instead?

Source

Why use a new subset split?

Strategy	Diversity	Expertise
One subset per tree	Very high	Often low
New subset per split	Moderate	High

- Using a new subset per split strikes a **balance** between
- Prevents trees from becoming too weak while still keeping

“Random selection of features at each node gives sub accuracy gains over bagging alone.”

Interim summary

- Random Forests reduce overfitting by **decorrelating**
- Two ingredients make this happen: Random data (bootstrap) and Random features (per split)
- Together, they build a strong yet diverse ensemble

iClicker Exercise 11.1

Select the most accurate option below.

- A. Every tree in a random forest uses a different bootstrap training set.
- B. To train a tree in a random forest, we first randomly select features. The tree is then restricted to only using those features.
- C. The `n_estimators` hyperparameter of random forest controls how many trees to build. More trees generally get a better performance on the validation or test set.
- D. In random forests we build trees in a sequential fashion. Each new tree is dependent upon the previous tree.
- E. Let classifiers A, B, and C have training errors of 10%, 20%, and 30% respectively. Then, the best possible training error for the ensemble of A, B, and C is 10%.

**Gradient boost
(high level)**

From random forests to

- In **random forests**, we build trees **independently** and combine their predictions
 - reduces **overfitting** by combining diverse models
- In **boosting**, we build trees **sequentially**
 - each new tree **learns from the mistakes** of the previous model
 - reduces **underfitting** by making the model smarter

Why it works

- **Sequential correction:** each tree improves on what's
- **Bias reduction:** starts simple and **adds complexity** g
- **Controlled learning:** small trees and a learning rate

Most commonly used ML models

- **XGBoost:** extremely optimized and widely adopted in
- **LightGBM:** faster training with large datasets
- **CatBoost:** handles categorical features efficiently

“More recently, gradient boosting machines (GBMs) have become a real knife in many a Kaggle’s toolbelt.”

— Sebastian Raschka, Joshua Patterson, & Corey Notman

Averaging and Stacking

Goal: Combining diverse models for better generalization

Averaging (Voting)

- Combines predictions from **multiple base models**
- Works for both **classification** and **regression**
- Improves stability and performance by averaging over
- Random forest is a type of averaging classifier

Types of Voting

Hard Voting

- Each model votes for a class label
- Final prediction = **majority vote**

Soft Voting

- Averages the probabilities from each model
- Final prediction = **average probabilities**
- Usually performs better

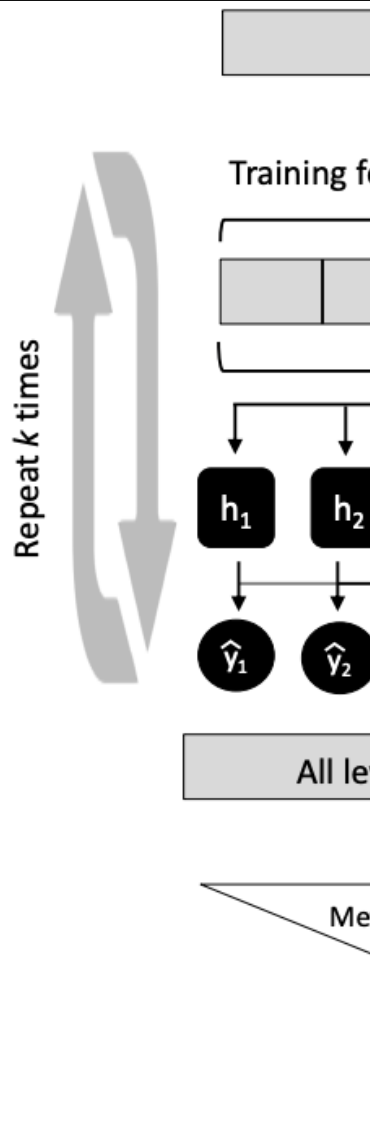
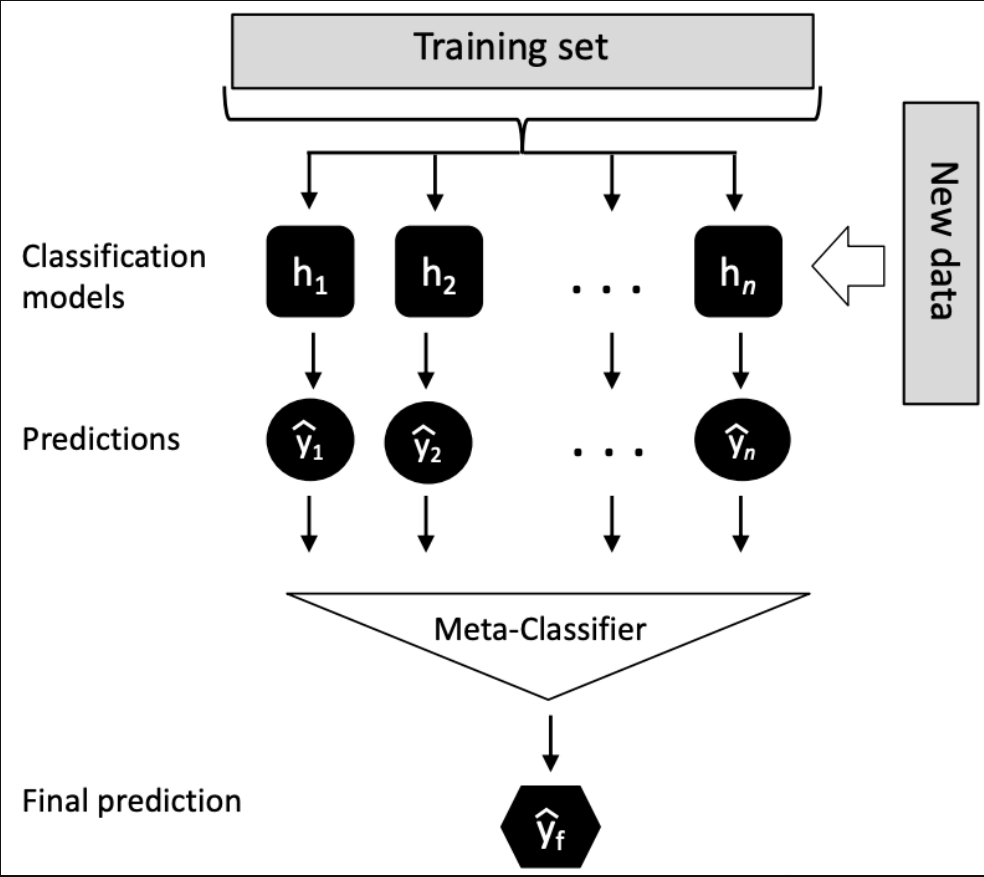
Stacking

- Goes one step beyond averaging
- Instead of simple averaging, trains a meta-model to learn how to combine base models' predictions

How stacking works

- Train several base models on the training data
- Collect their predictions on a validation set
- Train a meta-model on those predictions to learn the
- Final prediction = output of the meta-model

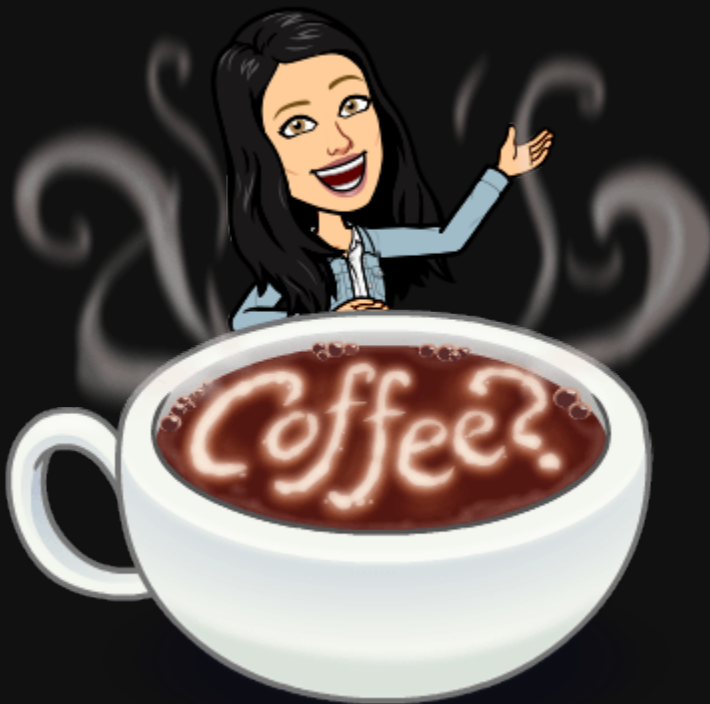
Stacking



Source

Break

Let's take a break!



/

Group Work: Class Demo

Coding

For this demo, each student should [click this link](#) to create accounts, then clone that repo locally to follow along with